

Malware Detection by Analysing Network Traffic with Neural Networks

Paul Prasse
University of Potsdam
Department of Computer Science
Potsdam, Germany
prasse@cs.uni-potsdam.de

Lukáš Machlica, Tomáš Pevný, Jiří Havelka
Cisco R&D Prague
Prague, Czech Republic
{lumachli, tpevny, jhavelka}@cisco.com

Tobias Scheffer
University of Potsdam
Department of Computer Science
Potsdam, Germany
scheffer@cs.uni-potsdam.de

Abstract—In order to evade network-traffic analysis, an increasing proportion of malware uses the encrypted HTTPS protocol. We study the problem of detecting malware on client computers based on HTTPS traffic analysis. Here, malware has to be detected based on the host address, timestamps, and data volume information of aggregated packets that are sent and received by all the applications on the client. We develop a scalable protocol that allows us to collect network flows of known malicious and benign applications as training data and derive a malware-detection method based on a neural language model and a long short-term memory (LSTM) network. We study the method's ability to detect new malware in a large-scale empirical study.

1. Introduction

Malware violates users' privacy, harvests passwords, is used to commit click-fraud, and can steal and encrypt users' files for ransom. Several different types of analysis are being used to detect malware, and because of the adversarial nature of the problem, no single approach offers perfect protection.

The analysis of an organization's network traffic complements decentralized antivirus software that runs on client computers. It allows organizations to enforce a security policy consistently throughout an entire network and to minimize the management overhead. This approach makes it possible to encapsulate malware detection into network devices or cloud services. In combination with client-based antivirus tools, network-traffic analysis can help to detect polymorphic malware [1] as well as new and as-yet unknown malware based on network-traffic patterns [2], [3].

An analysis of the HTTP payload can easily be prevented by using the encrypted HTTPS protocol. Currently, over 40% of the most popular websites support HTTPS [4]; Google, Facebook, LinkedIn, and many other popular sites encrypt their network traffic by default. In June 2016, an estimated 45% (and growing) fraction of all browser page loads are using HTTPS [5]. In order to continue to have an impact, traffic analysis has to work with HTTPS traffic.

On the application layer, HTTPS uses the HTTP protocol, but all messages are encrypted via the Transport Layer Security (TLS) protocol or its predecessor, the Secure

Sockets Layer (SSL) protocol. An observer can only see the client and host IP addresses and ports, and the timestamps and data volume of network flows. Via the Server Name Indication (SNI) protocol extension, an observer can also see the unencrypted host domain name. SNI is widely used because it enables clients to validate the certificate of servers that host multiple domains. The HTTP payload, including the HTTP header fields and the URL, are encrypted.

In this paper, we will develop a method that detects malware on client computers based on the observable information of HTTPS communication. The effectiveness of machine-learning approaches crucially depends on the availability of large amounts of labeled training data. We collect training and evaluation data using a VPN client that is able to observe the associations between executable files and HTTPS traffic on a large number of client computers.

In order to extract features from the domain name, we explore *neural language models* [6] which use neural networks to derive a low-dimensional embedding of a text. As a baseline, we study manually-engineered domain features [7]. We develop a malware-detection classifier based on *long short-term memory (LSTM)* networks [8]. LSTMs are a neural model for sequential data that account for long-term dependencies in information sequences. As reference method, we explore random forests.

This paper is organized as follows. Section 2 discusses related work. Section 3 describes our application environment, Section 4 our data. Section 5 describes the problem setting and Section 6 describe our method. Section 7 presents experiments; Section 8 concludes.

2. Related Work

Prior work on the analysis of *HTTP logs* [9] has addressed the problems of identifying command-and-control servers [10], unsupervised detection of malware [11], and supervised detection of malware using domain blacklists as labels [12], [7]. HTTP log files contain the full URL string, from which a wide array of informative features can be extracted [7]. In addition, each HTTP log file entry corresponds to a single HTTP request which also makes the timing and data volume information more informative than in the case of HTTPS, where the networking equipment

cannot identify the boundaries between requests that pass through the same TLS/SSL tunnel and log file entries may aggregate varying numbers of requests.

Prior work on *HTTPS logs* has aimed at identifying the application layer protocol [13], [14], [15], identifying applications that are hosted by web servers [16], and identifying servers that are contacted by malware [17]. Some methods process the complete sequence of TCP packets which is not usually recorded by available network devices. Lokoč et al. [17] use similar features to the ones that we use—that can easily be recorded for HTTPS traffic—and a similar method for generating labeled data based on a multitude of antivirus tools. However, they focus on the problem of identifying servers that are contacted by malware.

Prior work on neural networks for network-flow analysis [18] has worked with labels for client computers (infected and not infected)—which leads to a multi-instance learning problem. By contrast, our operating environment allows us to observe the association between flows and executable files. Prasse et al. [19] have explored HTTPS analysis using random forests in a preliminary report. LSTM networks [8] are widely used for translation, speech recognition and other natural-language processing tasks. Their ability to process sequential input and to account for long-term dependencies makes them appealing for the analysis of network flows. In computer security, their use has previously been explored for intrusion detection [20].

3. Operating Environment

This section describes our application and data-collection environment. In order to protect all client computers of an organization, a Cloud Web Security (CWS) service provides an interface between the organization's private network and the internet. Client computers establish a VPN connection to the CWS service. The service enforces the organization's security policy; it can block HTTP and HTTPS requests based on the host domain and on the organization's acceptable-use policy. The CWS service can issue warnings when it has detected malware on a client computer. Since it is expensive for administrators to process malware warnings, the proportion of false alarms among all issued warnings has to be small.

The CWS service can be configured to inspect HTTPS connections, provided that the service's root certificate has been installed as a trusted certificate on all client computers. This allows the CWS service to act as a man-in-the-middle between client and host, where it can decrypt and inspect HTTPS requests. This inspection can only be carried out for a small proportion of clients; we rely on it to collect training data. However, the deployed malware-detection model only uses observable features of HTTPS traffic and therefore does not require this interception. The VPN client software that runs on the client computer has access to the process table and the network interface. The VPN client can identify applications by means of a SHA hash code of their executable file, and can be configured to communicate the association between HTTP/HTTPS traffic and applications to the CWS

server. This enables signature-based malware detection, and it allows us to collect HTTPS traffic and associated hashes of the executable files that generate the traffic for inspected connections as training data.

The CWS service aggregates all TCP/IP packets between a single client computer, client port, host IP address, and host port that result from a single HTTP request or from the TLS/SSL tunnel of an HTTPS request into a network flow. This information is available for network devices that support the IPFIX [21] and NetFlow [22] formats. For each network flow, a line is written into the log file that includes data volume, timestamp, domain name, and duration information. For unencrypted HTTP traffic, this line also contains the full URL string; for HTTPS traffic, it includes the domain name—if that name is transmitted unencrypted via SNI—but not the URL string. The resulting sequence of log-file lines serves as input to the malware-detection model.

4. Data Collection

Since the proportion of HTTP traffic compared to HTTPS traffic declines continuously, we want to study malware detection in encrypted traffic. In our data collection, we therefore discard the URLs of HTTP requests from the log files, which leaves us only with information that would still be visible if all traffic were encrypted.

In order to obtain training and evaluation data, we have to label the network traffic by whether it originated from malicious or benign applications. For a number of organizations, we configure the VPN client to communicate a hash key of each executable file that initiates a network connection to the CWS server; the associated hash key is then saved with each flow in the log file. We label the traffic in retrospect, after the malware status of the files has been established by signature-based antivirus tools.

Virustotal.com is a web service that allows users to upload hash keys of executable files. The files are run through 60 antivirus solutions, and the results of this analysis are returned. We upload the hash keys of all executable files that have generated HTTP/HTTPS traffic to Virustotal; we label files as benign when the hash is known—that is, when the file has been run through the scanning tools—and none of the 60 scanning tools recognize the file as malicious. When three or more tools recognize the file as malicious, we label it as malicious. When only one or two virus scanners recognize the file as a virus, or when the hash key is unknown to Virustotal, we consider the malware status of the file *unknown*. We then label all traffic that has been generated by malicious executables as malicious, and all traffic of benign files as benign.

We collect three different data sets; we will refer to them as *current data*, *future data*, and an independent set of *training data for domain-name features*, based on the roles which these data sets will play in our experiments. The *current data* contains the complete HTTP and HTTPS traffic of 171 small to large computer networks that use the Cisco AnyConnect Secure Mobility Solution for a period of 5 days in July 2016. This data set contains 7,206,610 flows

TABLE 1. MALWARE FAMILIES

Malware family	Type	Variations
dealply	adware	506
softcnapp	adware, PUA	119
crossrider	adware, PUA	98
elex	adware, PUA	86
opencandy	adware, PUA	57
conduit	adware, spyware, PUA	56
browserfox	PUA	52
speedingupmypo	PUA	29
kraddare	adware, PUA	28
installcore	adware, PUA	27
mobogenie	PUA	26
pullupdate	PUA	25
iobit downloader	adware, PUA	24
asparnet	trojan, ransomware	24

of 43,272 distinct clients. The *future data* contains the complete HTTP and HTTPS traffic of 169 small to large different computer networks that use the Cisco AnyConnect Secure Mobility Solution for a period of 8 days in September 2016. The data set contains 10,159,990 flows of 38,296 distinct clients. The *training data for domain-name features* contains the HTTPS traffic of 21 small to large computer networks, collected over 14 days between February and April 2016. All data sets have been anonymized—information about organizations and user names have been replaced by random keys—and full URLs of HTTP requests have been removed.

The current data contains 23,093 unique hashes, 234,756 malicious and 6,971,854 benign flows; the future data 30,004 unique hashes, 703,360 malicious and 10,159,990 benign flows. Table 1 enumerates the types and frequency of different types of malware families, according to the classification of Virustotal.com. A large proportion of malware files is classified as *potentially unwanted applications* (PUAs). PUAs are usually bundled to freeware or shareware applications; depending on how a distributor chooses to employ these programs, they can change the browser settings and starting page, install tool bars and browser extensions, display advertisements, and can also reveal passwords to the attacker. They typically cannot be uninstalled without a virus removal tool or detailed technical background knowledge.

5. Client Malware Detection Problem

We will now establish the malware-detection problem setting. Our goal is to flag client computers that are hosting malware. Client computers are identified by a (local) IP address and a VPN user name.

For each interval of 24 hours, we count every client computer that establishes at least one network connection within that time interval as a separate *classification instance*. When at least one malicious application generates any network traffic, we label that instance as positive (malicious); otherwise the instance is negative (benign).

For each classification instance, malware detection model f has to decide whether to raise an alarm. The model observes a *sequence* $x_1, \dots, x_T$ of *flows* and raises an alarm

if the malware-detection score $f(x_1, \dots, x_T)$ exceeds some threshold τ . In each 24-hour interval, model f with threshold τ will detect some proportion of malware-infected clients, and may raise some false alarms. The trade-off between the number of detections and false alarms can be controlled by adjusting threshold τ . We will measure the following performance indicators.

- 1) *Recall* $R = \frac{n_{TP}}{n_{TP} + n_{FN}}$ is the proportion of n_{TP} , malicious instances that have been detected ($f \geq \tau$), relative to all (n_{TP} detected and n_{FN} undetected) malicious instances.
- 2) *Precision* $P = \frac{n_{TP}}{n_{TP} + n_{FP}}$ is the proportion of n_{TP} , malicious instances that have been detected, relative to all (n_{TP} malicious and n_{FP} benign) instances that have been flagged ($f \geq \tau$). The proportion of false alarms among all raised alarms is $1 - P$.
- 3) The recall at a specific precision, $R@x\%P$ quantifies the proportion of malicious instances that are detected when the threshold is adjusted such that at most $1 - x\%$ of all alarms are false alarms.
- 4) The precision-recall curve of decision function f shows the possible trade-offs that can be achieved by varying decision threshold τ ; decreasing τ tends to result in a higher recall and lower precision, increasing τ increases precision and decreases recall.

Note the difference between precision and the *false-positive rate*—the risk that a benign file is mistakenly classified as malware. For instance, at a false-positive rate of 10%, the expected number of false alarms equals 10% of the (typically huge) number of benign files for which a decision is made; hence, the false alarms would by far outnumber the actual malware detections. By contrast, at a precision of 90%, only 10% of all alarms would be false alarms.

Training data for model f consists of labeled sequences $S = \bigcup_{i=1}^n \{(x_{i1}, y_{i1}), \dots, (x_{iT_i}, y_{iT_i})\}$ in which each flow x_{it} is labeled by whether it has been sent or received by a malicious ($y_{it} = +1$) or benign ($y_{it} = -1$) application.

6. Network-Flow Analysis

This section presents our method that flags malware-infected client computers based on their network traffic.

6.1. Flow Features

The detection model processes *sequences of flows* $x_1, \dots, x_T$ sent to or received by one particular client computer. From each flow x_t , we extract a vector $\phi(x_t)$ that includes the log-transformed duration, log-transformed numbers of sent and received bytes, and the domain-name features that we will describe in Section 6.2.

6.2. Domain-Name Features

We explore several types of features that can be extracted from the host domain name.

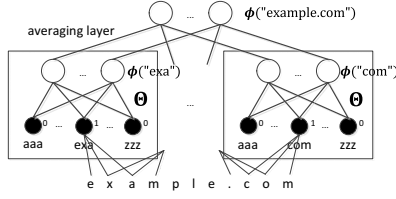


Figure 1. Neural language model network architecture

6.2.1. Engineered features. Franc et al. [12] develop a comprehensive set of 60 features of URL strings for malware detection in unencrypted HTTP traffic—here, the entire URL is visible to third parties. Their features include the ratio of vowel changes, the maximum occurrence ratio of individual characters for the domain and subdomain, the maximum length of substrings without vowels, the presence of non-base-64 characters, the ratio of non-letter characters, and many other characteristics. We extract the vector of these *engineered* domain-name features for all domains.

6.2.2. Character n -gram features. Character n -gram features decompose the domain string into sets of overlapping substrings; for instance, “example.com” is composed of the 3-grams “exa”, “xam”, “amp”, ..., “.co”, and “.com”. The number of n -grams that occur in URLs grows almost exponentially in n ; in our data, there are 1,583 character 2-grams, 54,436 character 3-grams, and 1,243,285 character 4-grams. If we added all character 3-gram features to the feature representation $\phi(x_t)$ of a flow, then the total number of features of an entire sequence of T flows would impose a heavy computational burden, and cause overfitting of the malware-detection model. In our experiments, we therefore explore character 2-gram features.

6.2.3. Neural domain-name features. We condense the character n -grams by means of a *neural language model*. Neural language models [6] derive low-dimensional, continuous-state representations of words which have the property that words which tend to appear in similar contexts have similar representations. Here, “words” are the overlapping character n -grams that constitute a domain name. We apply neural language models with the goal of finding a representation such that substrings that tend to co-occur in URL strings have a similar vector representation.

We use the neural-network architecture illustrated in Figure 1. The input to the network consists of character n -grams that are one-hot coded as a binary vector in which each dimension represents an n -gram; Figure 1 illustrates the case of $n = 3$. The input layer is fully connected to a hidden layer; we find that 100 hidden units give the best experimental results. The same weight matrix is applied to all input character n -grams. The activation of the hidden units is the vector-space representation of the input n -gram of characters. In order to infer the vector-space representation

of an entire domain-name, an “averaging layer” averages the hidden-unit activations of all its character n -grams.

Neural language models are trained to complete an initial part of a sentence, which forces the network to find a vector-space representation that allows to “guess” a word’s context from its vector-space representation. The “natural” reading order of a domain string is from the right to the left, because the domain ends with the most general part and starts with the most specific subdomain. During training, the network processes a sliding window of m overlapping n -grams in parallel, and averages the m resulting activation vectors of hidden units. The hidden layer is connected to an additional softmax output layer that again has one dimension for each possible character n -gram in a URL string. The output units infer a probability distribution over character n -grams. The output target during training is the character n -gram in the URL string that immediately precedes the input window of m overlapping n -grams.

We train the neural language model using all domain names in our *training data for domain-name features*, and the 500,000 domains that have the highest web traffic according to alexa.com. For each URL and each position of the sliding window, we perform a back-propagation update step for which the character that immediately precedes the input window serves as prediction target. We use the word2vec software package [23] with CBOW architecture. The neural language model generates a set of 100 host-domain features per flow.

6.3. Client Classifier

We develop a client classifier based on long-short term memory networks (LSTMs) and, as reference method, a classifier based on random forests. LSTMs process input sequences iteratively. Each LSTM unit has a memory cell which allows it to store results of inferences; the unit may refer to this memory cell in later time steps. This allows LSTMs to account for long-term dependencies in sequential input.

Both classifiers split each client’s traffic into sequences of 10 flows. After 10 flows $x_1, \dots, x_{10}$ have been observed for a client, the sequence is processed by the neural network. The feature representation of each flow (duration, bytes sent and received, and the neural host domain-name representation) is processed sequentially by a layer of 32 LSTM units. After processing the full input sequence, the LSTM layer passes its output to a layer of 128 cells with ReLU activation function, which in turn is connected to two softmax cells whose activation represents the decision-function scores of the classes malicious and benign. The softmax layer is trained with dropout regularization; that is, during training, the output of a randomly-drawn 50% of the 128 dense ReLU cells is set to zero in order to improve the robustness of the softmax layer.

During training, the target label for a sequence of 10 flows is positive if at least one of the 10 flows originates from a malicious application. At application time, the overall decision-function value $f(x_1, \dots, x_T)$ for the client’s full

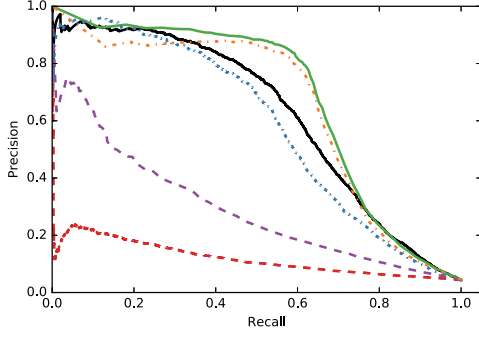


Figure 2. Precision-recall curve for malware detection on current data.

TABLE 2. DOMAIN CLASSIFICATION AND FEATURE TYPES

Feature type	R@70%P	R@80%P	R@90%P
neural	0.84	0.79	0.73
char 2-grams	0.83	0.76	0.62
engineered	0.68	0.36	0.0
neural+engineered	0.75	0.64	0.24

traffic over a 24-hour interval that is compared against threshold τ is the maximum activation of the positive output cell over all adjacent sequences of 10 flows.

For the random-forest classifier, the feature representation of 10 flows is stacked into a feature vector $\Phi(x_1, \dots, x_{10}) = [\phi(x_1) \dots \phi(x_{10})]^T$ which serves as input to the random-forest classifier. At training time, the target label of a sequence is positive if at least one flow originates from a malicious application, and at application time the decision-function value of the random forest is maximized over all 10-flow sequences.

7. Experiments

7.1. Classification of Host Domains

In our first experiment, we investigate the types of domain-name features with respect to their ability to distinguish between domains that are contacted by benign versus malicious applications. In this experiment, domains serve as classification instances; all domains that are contacted more often by malicious than by benign software are positive instances, and all other domains are negative instances. In our training data for domain features, there are 860,268 negative (benign) and 1,927 positive (malicious) domains.

We compare engineered, character 2-gram, and neural domain-name features as described in Section 6.2. We use 75% of the domains for training and 25% for testing; no domain is in both the training and the test data. We train a random forest classifier to discriminate positive from negative instances. For neural features, we find that the parameter combination of $n = 6$ and $m = 4$ (during training, the vector-space representations of 4 adjacent character 6-grams

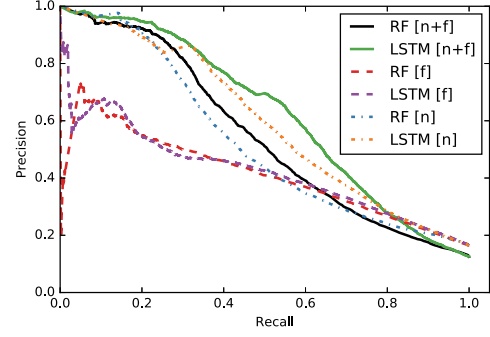


Figure 3. Precision-recall curve for malware detection on future data.

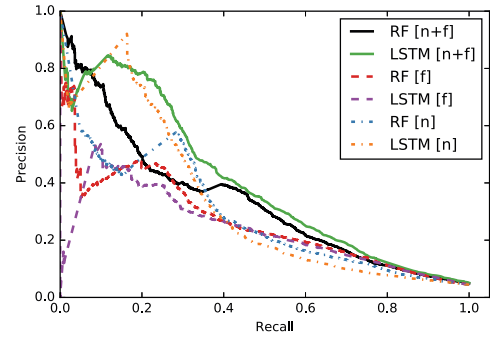


Figure 4. Precision-recall curves for unknown malware in future data.

are averaged) and $k = 100$ (the vector-space representation of a domain name has 100 dimensions) works best. Table 2 shows selected precision-recall values. We find that the neural domain-name features outperform both raw character 2-grams and the 60 engineered features. A combination of neural and engineered features performs worse than the neural features alone, which indicates that the engineered features inflate the feature space while not adding substantial additional information. Based on these findings, we continue to study the neural domain-name features and exclude the engineered and character 2-gram features in the following.

We observe that the domain names that are most confidently classified as malicious include subdomains of content delivery networks (*e.g.*, <https://...xx.atlas.cdn.yimg.com/>, <https://...xx.fbcdn.net/>), and automatically-generated domain names (*e.g.*, <https://s...vb-xl.net/>). The domains that are most confidently classified as benign span a full range of university and corporate home pages, news and entertainment, popular and less well-known services, and other “random” web pages.

7.2. Client Malware Detection

We will first report on experiments in which the client classifier is both trained and evaluated on *current data*. We

split the *current data set* into ten partitions with disjoint sets of users and conduct 10-fold cross validation. That is, we use nine partitions for training, evaluate the model on the tenth partition; we repeat this process with swapped partitions and average the performance indicators.

Figure 2 shows that the LSTM model consistently outperforms the random forest. The combination of neural domain-name features and numeric flow features outperforms both domain-name and flow features. The LSTM model detects 64% of malware at a precision of 70% (30% of all alarms are false alarms) or 41% at 90% precision.

Next, we train the model on the full *current data set* and evaluate it on the full *future data set*. Figure 3 shows that, again, the LSTM model outperforms the random forest, and that the combination of neural domain-name features and flow features excels over any single type of features. Here, the LSTM model detects 50% of all malware at 70% precision or 32% of all malware at 90% precision. Figure 4 shows that on malware with hashes that have first occurred *after* the training data have been recorded—which a signature-based detector would be unable to detect—the LSTM model detects 26% at a precision of 70% or 17% at a precision of 80%. Users are active for approximately 8 hours on average per 24-hour interval. The average time between the first flow sent by malware and its detection is 2.36 hours; in 52% of all detections, the malware is detected immediately after the first flow.

8. Conclusion

We can draw a number of conclusions from our study. A neural language model can transform a domain name into a low-dimensional feature representation that provides more information about whether the site is malicious than a set of carefully engineered features of domain-name characteristics. Our experimental setting allows us to collect large volumes of malicious and benign network traffic for model training and evaluation. The VPN client records the hash key of the executable file that has generated each flow, and by using VirusTotal we are able to determine, in retrospect, which flows originate from malware.

We have developed and studied a malware-detection model based on LSTMs that uses only observable aspects of HTTPS traffic. We conclude that the LSTM-based model outperforms random forests, and that the combination of neural domain-name features and numeric flow features outperforms either of these feature sets. This mechanism is able to identify a substantial proportion of malware in traffic that was recorded two months after the training data were recorded, including previously unseen malware. It complements signature-based and other behavior-based approaches to malware identification.

Acknowledgment. We would like to thank Virustotal.com for their kind support.

References

[1] M. E. Karim, A. Walenstein, A. Lakhotia, and L. Parida, "Malware phylogeny generation using permutations of code," *Journal in Com-*

puter Virology, vol. 1, no. 1-2, pp. 13–23, 2005.

[2] G. Gu, J. Zhang, and W. Lee, "Botsniffer: Detecting botnet command and control channels in network traffic," in *Proceedings of the Annual Network and Distributed System Security Symposium*, 2008.

[3] R. Perdisci, W. Lee, and N. Feamster, "Behavioral clustering of HTTP-based malware and signature generation using malicious network traces," in *Proceedings of the USENIX Conference on Networked Systems Design and Implementation*, 2010.

[4] "SSL Pulse," 2016, <https://www.trustworthyinternet.org/ssl-pulse/>.

[5] J. Aas, "Progress towards 100% HTTPS," *Let's Encrypt*, June 2016.

[6] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003.

[7] K. Bartos and M. Sofka, "Robust representation for domain adaptation in network security," in *Proceedings of the European conference on Machine Learning and Knowledge Discovery in Databases*, 2015.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, 1997.

[9] T. Nguyen and G. Armitage, "A survey of techniques for internet traffic classification using machine learning," *IEEE Communications Surveys, Tutorials*, vol. 10, no. 4, pp. 56–76, 2008.

[10] T. Nelms, R. Perdisci, and M. Ahamad, "Execscent: Mining for new c&c domains in live networks with adaptive control protocol templates," in *Proceedings of the USENIX Security Symposium*, 2013.

[11] J. Kohout and T. Pevny, "Unsupervised detection of malware in persistent web traffic," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing*, 2015.

[12] V. Franc, M. Sofka, and K. Bartos, "Learning detector of malicious network traffic from weak labels," in *Machine Learning and Knowledge Discovery in Databases*. Springer, 2015, pp. 85–99.

[13] C. V. Wright, F. Monrose, and G. M. Masson, "On inferring application protocol behaviors in encrypted network traffic," *Journal of Machine Learning Research*, vol. 7, pp. 2745–2769, 2006.

[14] M. Crotti, M. Dusi, F. Gringoli, and L. Salgarelli, "Traffic classification through simple statistical fingerprinting," *ACM SIGCOMM Computer Communication Review*, vol. 37, no. 1, pp. 5–16, 2007.

[15] M. Dusi, M. Crotti, F. Gringoli, and L. Salgarelli, "Tunnel hunter: Detecting application-layer tunnels with statistical fingerprinting," *Computer Networks*, vol. 53, no. 1, pp. 81–97, 2009.

[16] J. Kohout and T. Pevny, "Automatic discovery of web servers hosting similar applications," in *Proceedings of the IFIP/IEEE International Symposium on Integrated Network Management*, 2015.

[17] J. Lokoč, J. Kohout, P. Čech, T. Skopal, and T. Pevný, "k-NN classification of malware in HTTPS traffic using the metric space approach," in *Intelligence and Security Informatics*. Springer, 2016.

[18] T. Pevny and P. Somol, "Discriminative models for multi-instance problems with tree structure," in *Proceedings of the International Workshop on Artificial Intelligence for Computer Security*, 2016.

[19] P. Prasse, G. Gruben, L. Machlika, T. Pevny, M. Sofka, and T. Scheffer, "Malware detection by HTTPS traffic analysis," Technical Report. URN: urn:nbn:de:kobv:517-opus4-100942, 2017.

[20] R. Staudemeyer and C. Omlin, "Evaluating performance of long short-term memory recurrent neural networks on intrusion detection data," in *Proceedings of the South African Institute for Computer Scientists and Information Technologists Conference*, 2013.

[21] B. Claise, B. Trammell, and P. Aitken, "Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information," 2013, <https://tools.ietf.org/html/rfc7011>.

[22] Cisco Systems, "Cisco IOS netflow," <http://www.cisco.com/c/en/us/products/ios-nx-os-software/ios-netflow/index.html>, 2016.

[23] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems*, 2013.